

Détection de Tweets Suspects

Projet : Construction de Modèles et Déploiement

M2 FD&IA

Juin 2026

Introduction

Les réseaux sociaux sont une source majeure d'information mais aussi un vecteur de diffusion de contenus suspects (haineux, offensants, trompeurs). Ce projet vise à développer une solution de classification automatique capable d'identifier ces tweets suspects.

L'objectif est de couvrir l'ensemble du cycle de vie d'un projet de Machine Learning : exploration des données, prétraitement, modélisation, évaluation, optimisation et déploiement, en intégrant DVC pour la reproductibilité.

Dataset

Le jeu de données provient de Google Drive et contient **60 000 tweets** avec une étiquette binaire :

- **0** : Suspect
- **1** : Non suspect

Deux variables sont disponibles : message (texte du tweet) et label (classe).

Méthodologie

Prétraitement

Le nettoyage du texte est effectué via `src/preprocessing.py` et comprend les étapes suivantes :

1. **Minuscules** : uniformisation de la casse
2. **Suppression des URLs** : les liens n'ont pas de valeur sémantique
3. **Suppression des mentions** (@utilisateur) : identifiants uniques non généralisables
4. **Nettoyage des hashtags** : on garde le mot, on supprime le #
5. **Entités HTML** (&, <, etc.) : conversion en espaces
6. **Normalisation des répétitions** : goooooood → good
7. **Expansion des contractions** : don't → do not
8. **Suppression des caractères spéciaux** : seuls [a-zA-Z] sont conservés
9. **Suppression des stop words** : mots fréquents non discriminants (NLTK) + mots < 3 lettres
10. **Lemmatisation** avec WordNetLemmatizer et POS tagging : running → run

346 tweets vides après nettoyage sur 60 000 (supprimés).

Ces choix sont documentés dans le notebook d'analyse exploratoire (`notebooks/01_eda.ipynb`). La modélisation complète est détaillée dans `notebooks/02_modeling.ipynb` (consultable avec `uv run jupyter lab`).

Représentation des données

La représentation choisie est **TF-IDF** (Term Frequency : Inverse Document Frequency) avec un maximum de **5 000 features**. Ce choix se justifie par :

- Sa simplicité et son efficacité pour la classification de textes courts
- Sa capacité à pondérer l'importance des mots dans le corpus
- Sa compatibilité avec les modèles linéaires et naïfs bayésiens

Modèles utilisés

Trois algorithmes ont été implémentés et comparés :

1. Régression Logistique

Modèle linéaire avec `class_weight="balanced"` pour gérer le déséquilibre des classes. Limité à 1000 itérations pour assurer la convergence.

2. Naive Bayes (Multinomial)

Classifieur probabiliste adapté aux données de comptage (TF-IDF). Performant sur les textes mais sensible à la corrélation entre features.

3. Random Forest

Ensemble de 100 arbres de décision avec `class_weight="balanced"`. Capture les interactions non linéaires entre les mots.

Gestion du déséquilibre

La classe suspect est sous-représentée. La stratégie `class_weight="balanced"` ajuste les poids automatiquement. Une alternative (SMOTE) pourra être testée en optimisation.

Résultats

Performances des modèles

Modèle	Accuracy	Precision	Recall	F1-Score
Logistic Regression	97.04%	98.10%	98.61%	98.36%
Naive Bayes	92.76%	92.57%	99.95%	96.12%
Random Forest	97.54%	98.16%	99.12%	98.64%

Le **Random Forest** obtient les meilleures performances globales avec un F1-Score de **98.64%**.

Matrices de confusion

Matrices de confusion des 3 modèles

Matrices de confusion des 3 modèles

Lecture : VN (haut-gauche), FP (haut-droite), FN (bas-gauche), VP (bas-droite).

Courbes ROC et AUC

Courbes ROC des 3 modèles

Courbes ROC des 3 modèles

Les trois modèles présentent une AUC supérieure à 0.97, confirmant leur capacité de discrimination.

Validation croisée

La séparation train/test a été faite en **80/20 avec stratification** pour préserver la distribution des classes. Une validation croisée **5-fold** a été appliquée pour évaluer la stabilité des modèles (résultats dans notebooks/02_modeling.ipynb).

Pipeline DVC

Le pipeline reproductible se compose de 3 étapes :

```
uv run dvc repro
```

1. **preprocess** : nettoyage du texte
2. **train** : entraînement des 3 modèles avec TF-IDF
3. **evaluate** : calcul des métriques et génération des graphiques

Commandes essentielles

uv run dvc repro	# Exécuter le pipeline
uv run dvc status	# Vérifier l'état
uv run dvc checkout	# Restaurer depuis le cache
uv run dvc pull	# Récupérer depuis le remote
uv run dvc push	# Pousser vers le remote

Reproductibilité complète

```
git clone <repo-url>
cd tweet-suspect-detection
uv sync
uv run dvc pull
uv run dvc repro
```

Le dataset est téléchargeable via `uv run python src/download.py` (téléchargement interactif depuis Google Drive).

Optimisation

Une recherche d'hyperparamètres (Grid Search) a été effectuée sur deux modèles :

- **Régression Logistique** : $c \in \{0.01, 0.1, 1, 10, 100\}$: validation croisée 5-fold
- **Random Forest** : $n_estimators \in \{50, 100, 200\}$, $max_depth \in \{5, 10, 20, None\}$: validation croisée 3-fold

Les résultats détaillés sont visibles dans `notebooks/02_modeling.ipynb`.

Déploiement

L'application **Streamlit** est déployée localement et propose 3 onglets avec une interface interactive :

Prediction

Saisie d'un tweet et classification par les 3 modèles (Logistic Regression, Naive Bayes, Random Forest) avec score de confiance affiché sous forme de barre de progression. L'application affiche également un avertissement sur les limites du modèle (non-détection du harcèlement et du discours haineux). Des exemples prédéfinis permettent de tester rapidement l'application.

Interface Streamlit Prédiction Streamlit

Tableau de bord dynamique

Tableau de bord interactif généré avec **Plotly** permettant de : - Sélectionner un modèle parmi les 3 via un menu déroulant - Visualiser la matrice de confusion interactive (couleurs, survol) - Afficher les courbes ROC des 3 modèles superposées avec l'AUC - Comparer les performances (accuracy, precision, recall, F1) sous forme de barres groupées - Explorer l'importance des features (coefficients de la régression logistique ou importance de Gini pour Random Forest)

Tableau de bord dynamique

Tableau de bord dynamique

Historique

Toutes les analyses effectuées pendant la session sont conservées avec horodatage, tweet original, tweet nettoyé et résultats détaillés par modèle. L'historique peut être vidé manuellement.

Historique des analyses

Historique des analyses

Commande de lancement

```
uv run streamlit run src/deploy/streamlit_app.py
```

API FastAPI

Une interface API REST est également disponible comme alternative :

```
uv run uvicorn src.deploy.api:app --reload
```

Discussion

Limites

- La représentation TF-IDF ne capture pas le contexte sémantique
- Le dataset d'entraînement ne contient pas de tweets de harcèlement ou de discours haineux ; le modèle détecte principalement la négativité générale (plaintes, souffrance personnelle) et peut ne pas identifier correctement le racisme, les

insultes ciblées ou les menaces. Un avertissement est affiché dans l'interface Streamlit pour informer l'utilisateur de cette limitation

- Les modèles n'ont pas été testés sur des données réelles en streaming
- Pas de détection multilingue (stop words english uniquement)

Difficultés rencontrées

- Gestion des valeurs manquantes dans les tweets
- Déséquilibre des classes nécessitant des stratégies d'adaptation
- Mise en place du pipeline DVC avec téléchargement depuis Google Drive

Perspectives d'amélioration

- Utilisation de **Sentence Transformers** ou **BERT** pour une meilleure représentation sémantique
- Déploiement sur le cloud (Hugging Face Spaces / Render)
- Intégration CI/CD avec GitHub Actions pour automatiser le pipeline DVC
- Expérimentation avec MLflow pour le suivi des runs et la comparaison d'expériences
- Ajout d'un dataset spécialisé hate speech pour améliorer la détection des contenus sensibles
- Export CSV de l'historique des analyses
- Mode batch pour analyser un fichier CSV de tweets

Conclusion

Ce projet a permis de mettre en œuvre un pipeline complet de Machine Learning pour la détection de tweets suspects. Les meilleurs résultats ($F1 > 98\%$) montrent la faisabilité de la tâche avec des approches classiques. L'utilisation de DVC assure la reproductibilité du pipeline, et le déploiement via Streamlit/FastAPI permet une utilisation en conditions réelles.

Références

- Sujet : Détection_de_Tweet_Suspect_2026.md
- Code source : <https://github.com/...> (à compléter)
- Dataset : Google Drive